Single Sign-On Platform for MyDigital ID

SSO Integration Guideline Document

Document Owner	**My Digital ID Sdn. Bhd.**
Date	**12/2/2026**
Version	**6.0A**

Table of Contents

List of Tables

List of Figures

Acronyms and Abbreviations

Acronym/ Abbreviation	Definition
API	Application Programming Interface
DB	Database
DNS	Domain Name System
Https	HyperText Transfer Protocol Secure
ID	Identification
Idp	Secure Identity Provider
JWT	JSON Web Token
JSON	JavaScript Object Notation
OAuth	OAuth 2.0: An open-standard protocol for secure and delegated authorization.
OIDC	OpenID Connect
QR	Quick Response
RSC	Resource Security Code
SSO	Single Sign On
TLS	Transport Layer Security
TCP/ IP	Transmission Control Protocol/ Internet Protocol

Glossary/ Definition

Glossary	Definition
.NET MAUI	A cross-platform framework developed by Microsoft for building native applications for iOS, Android, macOS, and Windows using a single codebase.
3rd Party Application	Application that does not currently support MyDigital ID integration, but is the target for future MyDigital ID SSO implementation.
Client Component	External Third-Party Application client side.
Dart	A programming language developed by Google, designed for client-side development to build fast, modern apps for web, mobile, and desktop platforms.
Flutter	An open-source UI software development kit created by Google, used to develop cross-platform applications for Android, iOS, and web.
Ionic	An open-source framework for building cross-platform mobile applications using web technologies such as HTML, CSS, and JavaScript.
Java	A high-level, object-oriented programming language that is widely used for building cross-platform applications.
Keycloak Cluster (OIDC Provider)	Cloud application services consist of a group of interconnected computers that work together to perform identity and access management solutions.
Kotlin	An open-source programming language developed by JetBrains, designed to interoperate seamlessly with Java and run on the Java Virtual Machine (JVM).
MyDigital ID Mobile Application	Mobile Application contains MyDigital ID.
MyDigital ID Server	MyDigital ID Server that provides ID Authentication.
MyDigital ID SSO Server	MyDigital ID Single Sign On Server.
Node.js	A JavaScript runtime built on Chrome's V8 JavaScript engine, allowing developers to run JavaScript on the server side to build scalable, high-performance applications.
OAuth 2.0	An open-standard protocol for authorization that enables secure delegated access to resources without sharing credentials.
OpenID Connect (OIDC)	A protocol built on top of OAuth 2.0 that adds identity verification capabilities, allowing applications to confirm the identity of a user.
PHP	An open-source scripting language primarily designed for web development.

Glossary	Definition
Phyton	A popular JavaScript library for building user interfaces, particularly for single-page applications.
Progressive Web App (PWA)	A web-based application designed to function like a native app on any device, offering features such as offline functionality and push notifications.
React	A popular JavaScript library for building user interfaces, particularly for single-page applications.
Session	A temporary and secure connection between a user and the application, established after authentication and valid until the session expires or the user logs out.
Signet Cluster	Cloud application services consist of a group of interconnected computers that work together to perform MyDigital ID Authentication.
Swift	An open-source programming language developed by Apple for building applications across the Apple ecosystem, including **iOS**, **macOS**, **watchOS**, and **tvOS**
Third Party Application	A software application created by a developer or company intent to integrate with MyDigital ID.
Token	A digital artifact used to represent the authentication state of a user. Tokens are typically short-lived, digitally signed, and include claims such as user identity and expiration time.
User	The user in the client sends a request to the web server hosting the site.
Websocket Cluster	Cloud application services consist of a group of interconnected computers that work together to perform bidirectional communication with Signet Cluster.
WebAuthn	W3C standard API that enables passwordless, phishing-resistant, and secure multi-factor authentication (MFA) across web browsers and platform

SECTION 1: SYSTEM OVERVIEW

1.0 System Overview

1.1 SSO Integration

SSO Integration uses Keycloak, an open-source Identity and Access Management (IAM) for modern apps like single-page applications, mobile apps, and REST APIs. The services provided to facilitate the integration of SSO include:

- Authentication: Centralized login service to verify user identities
- Authorization: Role-based and implements detailed access control for applications
- Token Management: Issuance and validation of tokens using OpenID Connect protocol.
- User Federation: Integration with external user directories like LDAP, Active Directory and more.

1.2 System Architecture

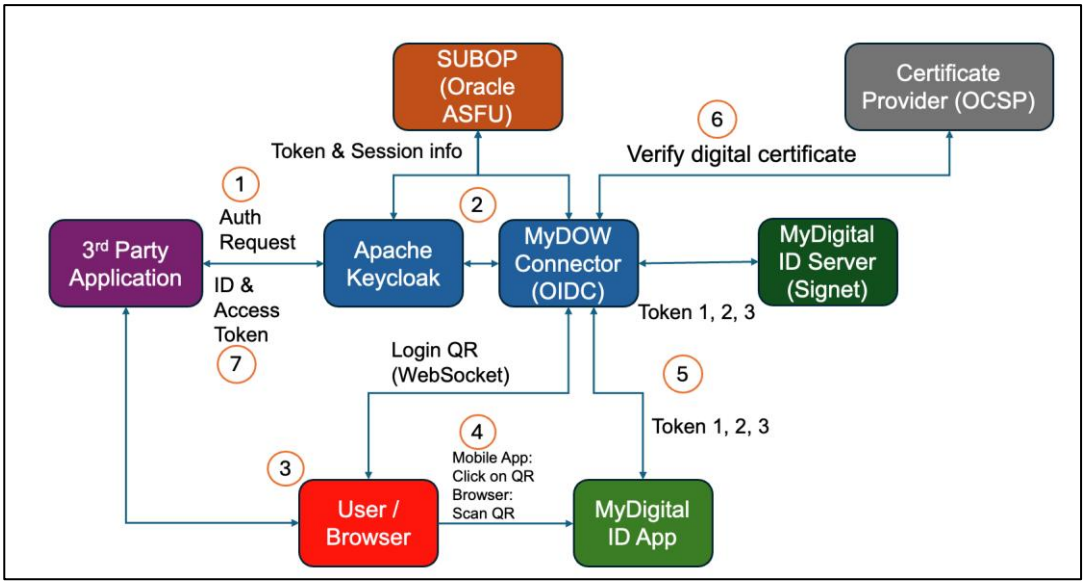

Figure 1: System Architecture

1.2.1 System Architecture Flow

1. The user will go to the Login Page in the 3rd party application. The 3rd party application will be redirected to the Keycloak.
2. Keycloak then will be redirected to MyDOW Connector (OIDC).
3. MyDOW Connector will generate a unique QR Code and will display it to the user.
4. The user clicks the QR Code (mobile) or scans the QR Code (browser) and authenticates using the MyDigital ID application.
5. MyDigital ID application connects to MyDOW Connector and performs 3-way handshaking.
6. On the 3rd handshake, MyDOW Connector will receive the user certificate and check the certificate with OCSP.
7. MyDOW connector will return the auth token to Keycloak and Keycloak will return the token to the 3rd party application.

1.3 MyDigital ID SSO Platform

MyDigital ID SSO platform consists of the Identity and Access Management (IAM) platform on the frontend that supports OAuth 2.0 and OpenID Connect. Keycloak provides the functionality to manage authentication client secret, authentication flow behavior and other operational elements that is required by OAuth 2.0 and OIDC to function.

MyDOW OIDC Connector plugin is a purpose-built plugin that was developed to interface OIDC with the mechanism that performs 3-way handshaking of MyDigital ID (refer to MyDigital ID Integration Guideline Document). In essence, MyDOW OIDC connector is presented as an Identity Provider to Keycloak.

1.4 Keycloak SSO Protocol

Keycloak uses industry-standard protocols like OAuth 2.0 and OpenID Connect secure authentication and authorization. Using industry-standard protocols is crucial not only for ensuring robust security but also for simplifying integration with both existing and new applications.

1.4.1 OAuth 2.0

With OAuth 2.0, sharing user data to third-party applications is easy, does not require sharing user credentials, and allows control over what data is shared. Four (4) roles defined in OAuth 2.0:

- **Resource owner**: The end user that owns resources that an application wants to access.
- **Resource server**: The service hosting the protected resources.
- **Client**: The application that would like to access the resource.
- **Authorization server**: The server issuing access to the client, which is the role of Keycloak.

In an OAuth 2.0 protocol flow, the client requests access to a resource on behalf of a resource owner from the authorization server. The authorization server issues limited access to the resource at the resource server by including access token in the request.

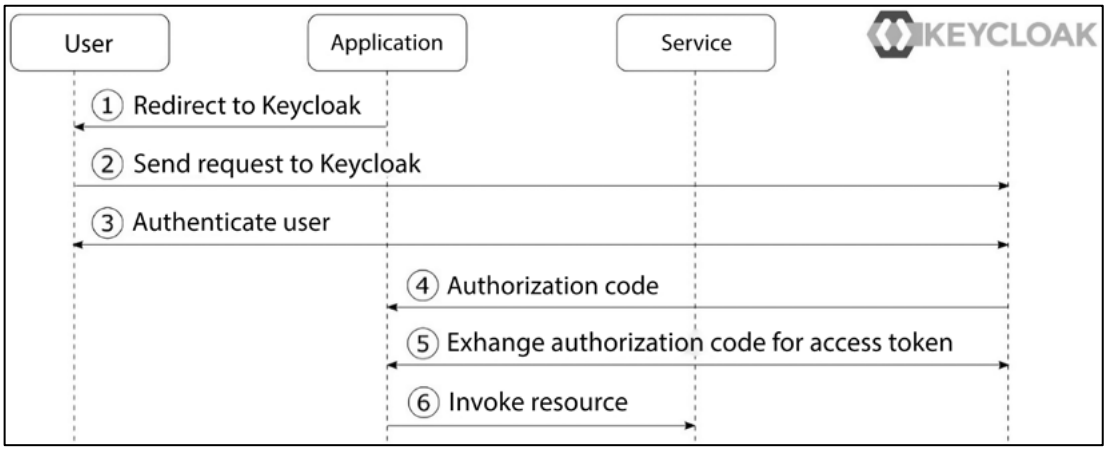

Figure 2: Simplified OAuth 2.0 Authorization Code Grant Type

The steps in the diagram are as follows:

1. The application sends an authentication request to the user's browser to be redirected to Keycloak for authorization.
2. The browser redirects the user to Keycloak's authorization page.
3. If the user is not authenticated with Keycloak, Keycloak authenticates the user
4. The application gets an authorization code from Keycloak.
5. The application then exchanges the code for an access token from Keycloak.
6. The application uses the access token to access the protected resource.

Access tokens are passed around from the application to services, usually having a short lifetime. To get new access tokens without repeating the whole process, a refresh token is used.

1.4.2 OpenID Connect

Since OAuth 2.0 is for authorization, but not authentication, OpenID Connect adds an authentication layer on top of OAuth 2.0. OpenID Connect has enabled a whole ecosystem of websites to no longer need to deal with user management and authenticating users. OpenID Connect is useful to have a centralized solution for authentication supporting single sign-on and increases security as applications do not have access to the user credentials directly. Furthermore, it enables the use of stronger authentication, such as OTP or WebAuthn, without the need to support it directly within applications.

OpenID Connect (OIDC) involves three (3) main roles in its protocol:

1. **End user**: It is the same as the resource owner in OAuth 2.0. This is simply the human being who is logging in or being authenticated.
2. **Relying Party (RP):** The application or website that needs to verify or authenticate the end user.
3. **OpenID Provider (OP):** The identity provider that is authenticating the user, which is the role of Keycloak.

In OpenID Connect, the RP asks the OP for the user's identity and can also obtain a token since it builds on OAuth 2.0. OpenID Connect uses OAuth 2.0's Authorization Code grant but adds `scope=openid` to make it an **authentication request** instead of just an **authorization request**. There are two (2) flows in OpenID Connect:

1. Authorization code flow: This follows the OAuth 2.0 Authorization Code flow, returning an authorization code that can be exchanged for an ID token, access token, and refresh token.
2. Hybrid flow: The ID token and authorization code are returned together in the initial request.

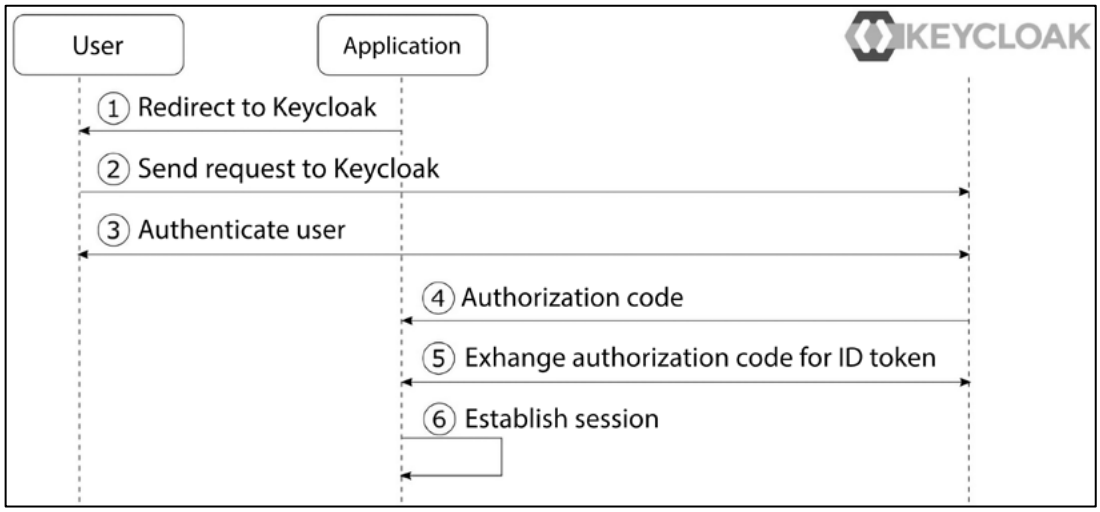

Figure 3: Simplified OpenID Connect Authorization Code Flow

The steps in the diagram are as follows:

1. The app prepares a request to authenticate and asks the user's browser to be redirected to Keycloak.
2. The browser redirects the user to Keycloak's login page (authorization endpoint).
3. If the user is not logged in yet, Keycloak will ask them to log in.
4. The app gets an authorization code from Keycloak as a response.
5. The app then exchanges this authorization code with Keycloak for an ID token and an access token.
6. The app uses the ID token to identify the user and start a logged-in session.

1.5 MyDigital ID SSO Protocol Sequence

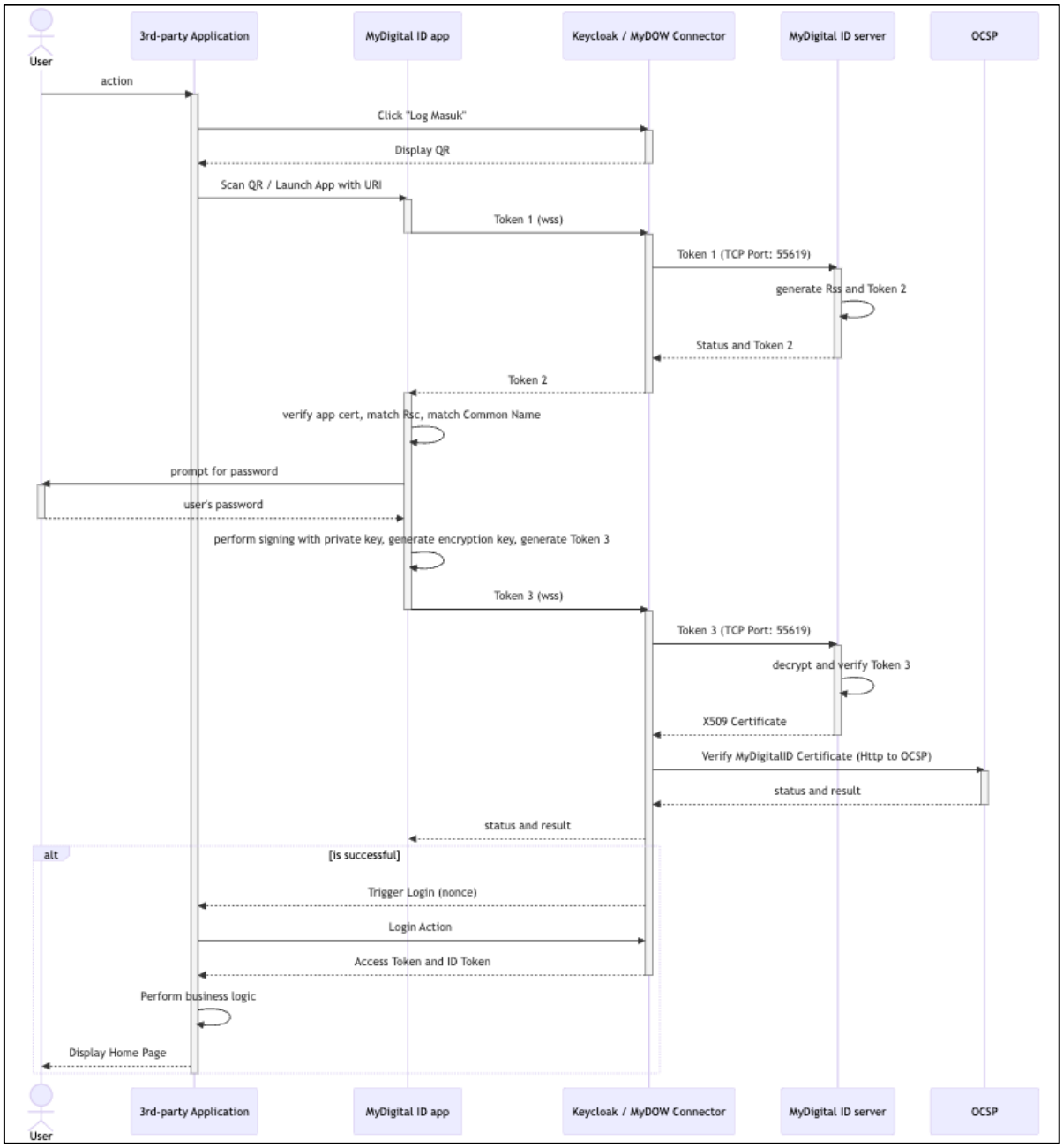

Figure 4: MyDigital ID SSO Protocol Sequence Diagram

1.5.1 MyDigital ID SSO Protocol Sequence Components

Components:

1. **User**: The end-user requesting access to resources.
2. **3rd-Party Application (App)**: The client application consists of a mobile application or web application that the user interacts with.
3. **MyDigitalID App**: An app responsible for verifying users and generating tokens.
4. **Keycloak/MyDOW Cluster (OIDC Provider)**: The OpenID Connect (OIDC) provider for handling authentication.
5. **WebSocket Cluster (WSS)**: Manages WebSocket connections for real-time communication.
6. **Signet Cluster**: Handles secure token verification and status updates.
7. **Flow Explanation:**
    1. **User Request**
        - The user makes a request to the application for a resource, but there is no prior authentication.
    2. **Redirect to Keycloak**
        - The application redirects the user to Keycloak for authentication.
        - Keycloak detects the MyDigital ID Identity Provider as the IDP stated in the authentication flow of the client.
        - Keycloak redirects requests to MyDigital MyDOW IDP which then produces the unique nonce and generates the MyDigital ID QR code to be scanned and application URL link to Signet server if the QR is clicked.
    3. **Action and Token Generation**
        - The user performs an action by scanning using MyDigital ID app (Desktop web app) or click on the QR code (Mobile app or Mobile PWA).
        - MyDigital ID commence the MyDigital ID authentication protocol (3-way handshaking).
    4. **WebSocket Initiation**
        - While the MyDigital ID app is bound to the WebSocket server, the 3rd party application is also connected with the WebSocket

Connections. The connections from the 3rd party application will join a channel determined by the nonce.

- The Two apps, 3rd party login and my Digital ID WebSocket login are bound by the nonce WebSocket channel.

5. **Final Token Generation and Authentication**
    - Once the 3-way handshaking process is accomplished (includes the password verification), a JWT Token is responded to Keycloak.
    - The JWT Token responded by MyDOW OIDC to Keycloak will consist of full name and IC Number.
    - Once Keycloak receives the JWT Token, Keycloak will take the claims (fullname and IC Number) and incorporate the info into a newly generated JWT token to be returned to the initiator client.
6. **Resource Access**
    - The user accesses the application with the JWT token.
    - The application validates the token with Keycloak.
    - Upon successful validation, Keycloak informs the application, and the application provides access to the requested resources.

1.6 MyDigital ID SSO Keycloak Client

A Keycloak client is an application or service that interacts with Keycloak for authentication and authorization. In Keycloak, a "client" can be any system that integrates with Keycloak to use its identity management capabilities, such as single sign-on (SSO), user authentication, and role-based access control (RBAC).

Keycloak clients are typically web applications, mobile applications, or even backend services that use Keycloak to authenticate users and obtain access tokens, or they could be microservices that need to secure their APIs.

1.6.1 MyDigital ID SSO Keycloak Client Configuration

When configuring a Keycloak client, you usually specify the following settings:

1. **Client ID**: A unique identifier for the client within Keycloak. This is how Keycloak knows which client is making a request.
2. **Client Secret**: A secret key used for secure communication between the client and Keycloak (in the case of confidential clients).
3. **Redirect URI**: The URL to which Keycloak will redirect the user after successful authentication. This is typically the URL of the application that the user is trying to access.
4. **Protocol**: The authentication protocol used by the client. Common protocols include:
    - **OpenID Connect** (OIDC) – A modern protocol based on OAuth 2.0, commonly used for web and mobile applications.
    - **OAuth 2.0** – A framework for access delegation, widely used for authorization.
5. **Access Type**:
    - **Confidential**: Clients that can securely store credentials (e.g., server-side applications). These clients authenticate themselves to Keycloak using client secrets.
    - **Public**: Clients that cannot securely store credentials (e.g., single-page applications, mobile apps).
    - **Bearer-only**: Clients that only accept access tokens and cannot initiate authentication themselves. Common for APIs or resource servers.
6. **Roles and Scopes**: The permissions or access levels that the client can request, often used to limit or specify access to certain resources or actions.

1.6.2 Client Flow in MyDigital ID SSO Keycloak

1. A user accesses a client application and is redirected to Keycloak for authentication.
2. Keycloak authenticates the user (e.g., through login credentials, social login, etc.).
3. After successful authentication, Keycloak sends an authorization code or access token back to the client application (depending on the flow used).
4. The client can use the token to make authorized API calls, or the token can be stored for the duration of the session.

1.6.3 Example of Client Flow in MyDigital ID SSO Keycloak Use Cases

- **Web Application**: A client in Keycloak could represent a React or Angular app that interacts with a backend API to fetch user data after authentication.
- **Mobile App**: A client could represent a mobile app that needs to authenticate users via Keycloak and interact with a REST API.
- **Service-to-Service Communication**: A client could also be a backend service that needs to authenticate itself when calling other services or APIs.

In summary, a MyDigital ID SSO Keycloak client is a representation of an application or service that uses Keycloak for handling authentication and authorization, with various settings and protocols to define its interaction with Keycloak's identity management features.

SECTION 2: WEB LOGIN, MOBILE APP SSO INTEGRATION AND DEVELOPER GUIDE

2.0 3rd Party Application Integration with MyDigital ID Keycloak SSO Integration

Integration with MyDigital ID Keycloak SSO is usually done by incorporating standard open source libraries that are readily available on various platforms. These libraries are usually maintained by the open source community, most often up to date with features and security patches. This method is recommended because it will reduce development time and be more stable due to the fact that they are being used widely by the community. The following section will outline the necessary task required to accomplish such integration with examples towards the end.

2.1 MyDigital ID SSO Integration Checklist

Below is a list of information(s) that is needed for us to assess the compatibility and support feature of the current 3rd party application with the MyDigital ID SSO platform. Assessment is primarily needed to assess if the current web or application framework has a standard Keycloak library that will reduce development time or custom development for the authentication protocols aforementioned which will extend development time.

Table 1: Application Survey Form

#	Item	Description
1	Application Survey Form	A third-party application product owner MUST provide us the details of the software framework used and send Application form email to sso@myid.my to request the integration. The Application Survey Form should include the following details: • Apps: • Language: • Language Version: • Framework: • Framework Version: • Os: • Os Version: • DB: • DB Version: • Authentication Type: • Public App/Internal App: Warganegara /Bukan Warganegara / Staff/Orang Awam **Please refer Appendix A**

2.2 Library for Keycloak SSO Integration

Below is a list of libraries that supports Keycloak SSO Integration with the suitable Programming Language and framework. The website or application environment must meet the required language and framework versions before installing the library.

Table 2: List of Libraries for SSO Integration

Language	Language Version	Framework	Framework Version	Library	Library Version
Node.js	12.x, 14.x	Express.js	4.x	keycloak-connect	11.x
	16.x+	Express.js	4.x	keycloak-connect	12.x
	14.x+	NestJS	7.x to 9.x	nest-keycloak-connect	2.x

Language	Language Version	Framework	Framework Version	Library	Library Version
	16.x+	NestJS	10.x	nest-keycloak-connect	3.x
Java	8+	Spring Boot	2.x	spring-boot-starter-oauth2-client	2.x
	11+	Spring Boot	2.x	spring-security-oauth2	5.x
	8+	Java EE (Jakarta EE)	8.x	keycloak-java-adapter	1.x
	4+	Quarkus	1.x	keycloak-authorization	1.x
Python	3.6+	Flask	1.x to 2.x	flask-oauthlib	0.9
	3.6+	Django	3.x	django-oauth-toolkit	1.x
	3.6+	FastAPI	0.68+	fastapi-keycloak	1.x
PHP	8.1+	Laravel	10.x	vizir/laravel-keycloak-web-guard	4.1.0
	8.0+	Laravel	8.x to 9.x	vizir/laravel-keycloak-web-guard	4.1.0
	7.3 - 7.4	Laravel	8.x	vizir/laravel-keycloak-web-guard	4.1.0
	8.1+	Laravel	10.x	laravel-keycloak-guard	3.x
	8.0+	Laravel	9.x	laravel-keycloak-guard	2.x
	7.4+	Laravel	7.x to 8.x	laravel-keycloak-guard	2.x
	7.0+	Codeigniter	3.x	oauth2-client	1.x
	7.4+	Codeigniter	3.x	oauth2-client	2.x
	8.0+	Codeigniter	4.x	openid-connect-php	3.x
	7.4+	Symfony	4.x	knpuniversity/oauth2-client	2.x
React	17.x+	Next.js	10.x+	next-auth	3.x+
	16.x+	Gatsby	3.x+	gatsby-plugin-keycloak	1.x
.NET MAUI	6.x+	Xamarin	5.x+	Xamarin.Auth	1.x+
Ionic	6.x+	Angular	11.x+	angular-oauth2-oidc	11.x+

Language	Language Version	Framework	Framework Version	Library	Library Version
Swift	5.x+	SwiftUI	2.x+	OAuthSwift	2.x+
Kotlin	1.4+	Spring Boot	2.x+	spring-security-oauth2-client	5.x+
	1.4+	Ktor	2.x+	ktor-client-auth	2.x+
Dart	2.x+	Flutter	2.x+	flutter_appauth	0.9.x+
	2.x+	Flutter	3.x+	oauth2	2.x+
	2.x+	Flutter	3.x+	keycloak_wrapper	1.x+
	2.x+	AngularDart	6.x+	angular_oauth2	6.x+
	2.x+	Aqueduct	4.x+	aqueduct_auth	4.x+
Go	1.22.x	Gin	1.x+	golang.org/x/oauth2	0.x+
	1.22.x	Gin	1.x+	github.com/coreos/go-oidc/v3/oidc	3.x+
	1.22.x	Echo	4.x+	github.com/coreos/go-oidc/v3/oidc	3.x+
	1.22.x	Fiber	2.x+	golang.org/x/oauth2	0.x+

2.3 Keycloak Configuration

The configuration of Keycloak for an application or website is managed through the Keycloak Admin Console, which provides a user-friendly interface for setting up and maintaining Keycloak's integration. This console allows administrators to configure various components necessary for the secure authentication and authorization of users. Below is a comprehensive list of configurations required to integrate Keycloak into the application or website, ensuring that user access is properly controlled and authenticated in accordance with the desired security policies.

Table 3: Keycloak Configuration Example

Field	Description
KEYCLOAK_CLIENT_ID	<Client_ID>
KEYCLOACK_CLIENT_SECRET	xxxxxxxxxxxxxxxxxxxxxxxxxx

KEYCLOACK_REDIRECT_URI	<3rd Party App URL to handle authorization code>
KEYCLOACK_BASE_URL	<MyDigital ID SSO DNS host>
KEYCLOCK_REALM	<Realm>
KEYCLOACK_URL_AUTHORIZE	https:// <MyDigital ID SSO DNS host> /realms/ <Realm> /protocol/openid-connect/auth
KEYCLOACK_URL_ACCESS_TOKEN	https:// <MyDigital ID SSO DNS host> /realms/ <Realm> /protocol/openid-connect/token
KEYCLOACK_URL_USER_INFO	https:// <MyDigital ID SSO DNS host> /realms/ <Realm> /protocol/openid-connect/userinfo

2.3.1 Endpoint

Table 4: Endpoint List

Endpoint Name	Description	Example URL: MyDigital ID SSO DNS host
Authorization Endpoint	Used to obtain an authorization code or implicit access token via user authentication	https:// <MyDigital ID SSO DNS host> /realms/ <Realm> /protocol/openid-connect/auth
Token Endpoint	Used to exchange authorization codes, refresh tokens, or client credentials for access tokens	https:// <MyDigital ID SSO DNS host> /realms/ <Realm> /protocol/openid-connect/token
Userinfo Endpoint	Returns claims (user profile information) about the authenticated user.	https:// <MyDigital ID SSO DNS host> /realms/ <Realm> /protocol/openid-connect/userinfo
Logout Endpoint	Allows users to securely log out by terminating their session and revoking authentication tokens, using parameters such as the ID token hint, client ID, and post-logout redirect URI to ensure proper session closure and redirection.	https:// <MyDigital ID SSO DNS host> /realms/ <Realm> /protocol/openid-connect/logout?client_id= <Client_ID> &id_token_hint= <ID Token> &post_logout_redirect_uri=<Home URL>
JWKS Endpoint	Provides the public keys used to verify JSON Web Tokens (JWTs)	https:// <MyDigital ID SSO DNS host> /realms/ <Realm> /protocol/openid-connect/certs

2.4 Developer Guide for SSO Integration

2.4.1 PHP 7 with Laravel Framework

This configuration guide is specifically applicable to **PHP v7.4.33** and **Laravel framework v8.83.29**. It is important to note that this guide may not be universally compatible with all PHP or Laravel versions. Developers should verify that their environment, including libraries, programming language, and framework versions, aligns with these requirements to ensure successful implementation.

2.4.1.1 Environment Configuration

Keycloak is configured in the .**env** file. This allows easy management of Keycloak configuration without hardcoding sensitive information in the application code:

```
KEYCLOAK_BASE_URL=MydigitalID_host

KEYCLOAK_REALM=mydid

KEYCLOAK_CLIENT_ID=Client ID

KEYCLOAK_CLIENT_SECRET=NUxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

KEYCLOAK_REDIRECT_URI=http://127.0.0.1:8000/callback
```

Figure 5: Environment Configuration in PHP Laravel

2.4.1.2 Library Installation

Install the Keycloak Guard library in the Laravel project folder.

```
composer require vizir/laravel-keycloak-web-guard:4.1.0
```

Figure 6: Keycloak Web Guard Library Installation

2.4.1.3 Publish Configuration File

Publish the configuration file will create a **keycloak-web.php** file in config directory, which can edit to customize settings like routes and redirect URLs.

```
php artisan vendor:publish --provider="Vizir\KeycloakWebGuard\KeycloakWebGuardServiceProvider"
```

Figure 7: Publish keycloak-web.php File

2.4.1.4 Define Scopes

In **keycloak-web.php** file, define scopes as 'openid'.

```
'scopes' => ['openid'],
```

Figure 8: OpenID Connect (OIDC) Scopes

2.4.1.5 Add Keycloak Driver

Add the configuration for the Keycloak driver in the **auth.php** file to define Keycloak as an authentication provider for the application.

```
'guards' => [
    'web' => [
        'driver' => 'keycloak-web',
        'provider' => 'users',
    ],
],
```

Figure 9: Keycloak Guards Driver

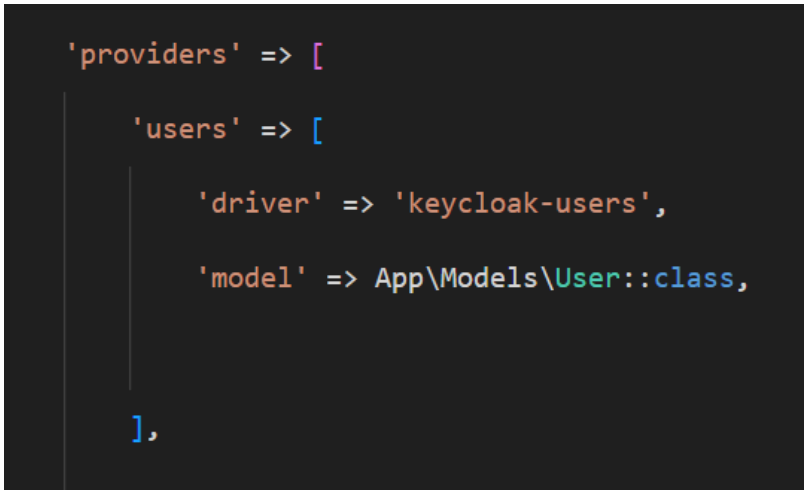

Figure 10: Keycloak Providers Driver

2.4.1.6 Add Column in Model

If the application or website does not store the name and IC number in the database, the developer can add the ‘nama’ and ‘nric’ column to the protected fillable section in the **User.php** file. This approach ensures that the name and IC number is securely managed and can be updated within the user model while maintaining data integrity.

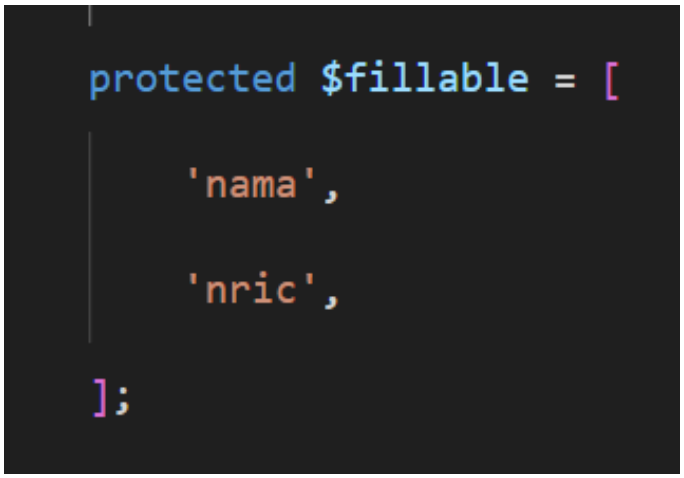

Figure 11: Add ‘nama’ and ‘nric’ Column

2.4.1.7 Add New Routes

Add the new routes for integration with MyDigital ID SSO in the **web.php** file to ensure proper routing for actions such as account login, callback and logout. Protected main route within the keycloak-web middleware to ensure that only authenticated users can access it.

```php
use Illuminate\Support\Facades\Route;

use Vizir\KeycloakWebGuard\Controllers\AuthController;


Route::get('/', function () {
    return view('welcome');
});


Route::group(['middleware' => 'keycloak-web'], function () {
    Route::get('/dashboard', function () {
        return view('dashboard');
    })->name('dashboard');
});


Route::get('/mydid-login', [AuthController::class, 'login'])->name('keycloak.login');
Route::get('/mydid-callback', [AuthController::class, 'callback'])->name('keycloak.callback');
Route::get('/mydid-logout', [AuthController::class, 'logout'])->name('keycloak.logout');
```

Figure 12: Add New Routes

2.4.1.8 Call Route Name

Call the route name that already defined in **web.php** to use in views.

```
<button onclick="window.location.href='{{ route('keycloak.logout') }}'" class="button-1">Logout</button>

@else

<div class="card">

    <h2>No user information available.</h2>

    <button onclick="window.location.href='{{ route('keycloak.login') }}'" class="button-1">Login</button>
```

Figure 13: Call Route Name

2.4.2 PHP 8 with Laravel Framework

This configuration guide is specifically applicable to **PHP v8.1.25** and **Laravel framework v10.48.12**. It is important to note that this guide may not be universally compatible with all PHP or Laravel versions. Developers should verify that their environment, including libraries, programming language, and framework versions, aligns with these requirements to ensure successful implementation.

2.4.2.1 Environment Configuration

Keycloak is configured in the .**env** file. This allows easy management of Keycloak configuration without hardcoding sensitive information in the application code:

```
KEYCLOAK_BASE_URL=MydigitalID_host

KEYCLOAK_REALM=mydid

KEYCLOAK_CLIENT_ID=Client ID

KEYCLOAK_CLIENT_SECRET=NUxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

KEYCLOAK_REDIRECT_URI=http://127.0.0.1:8000/callback
```

Figure 14: Environment Configuration in PHP Laravel

2.4.2.2 Library Installation

Install the Keycloak Guard library in the Laravel project folder.

```
composer require vizir/laravel-keycloak-web-guard:4.1.0
```

Figure 15: Keycloak Web Guard Library Installation

2.4.2.3 Publish Configuration File

Publish the configuration file will create a **keycloak-web.php** file in config directory, which can edit to customize settings like routes and redirect URLs.

```
php artisan vendor:publish --provider="Vizir\KeycloakWebGuard\KeycloakWebGuardServiceProvider"
```

Figure 16: Publish keycloak-web.php File

2.4.2.4 Define Scopes

In **keycloak-web.php** file, define scopes as ‘openid’.

```
'scopes' => ['openid'],
```

Figure 17: OpenID Connect (OIDC) Scopes

2.4.2.5 Add Keycloak Driver

Add the configuration for the Keycloak driver in the **auth.php** file to define Keycloak as an authentication provider for the application.

```
'guards' => [
    'web' => [
        'driver' => 'keycloak-web',
        'provider' => 'keycloak_users',
    ],
    'session' => [
    'driver' => 'session',
    'provider' => 'users',
    ],
],
```

Figure 18: Keycloak Guards Driver

```
'providers' => [
    'users' => [
        'driver' => 'eloquent',
        'model' => App\Models\User::class,
    ],

    'keycloak_users' => [
        'driver' => 'keycloak-users',
        'model' => App\Models\User::class,
    ],
],
```

Figure 19: Keycloak Providers Driver

2.4.2.6 Add Column in Model

If the application or website does not store the name and IC number in the database, the developer can add the ‘nama and ‘nric column to the protected fillable section in the User.php file. This approach ensures that the name and IC number is securely managed and can be updated within the user model while maintaining data integrity.

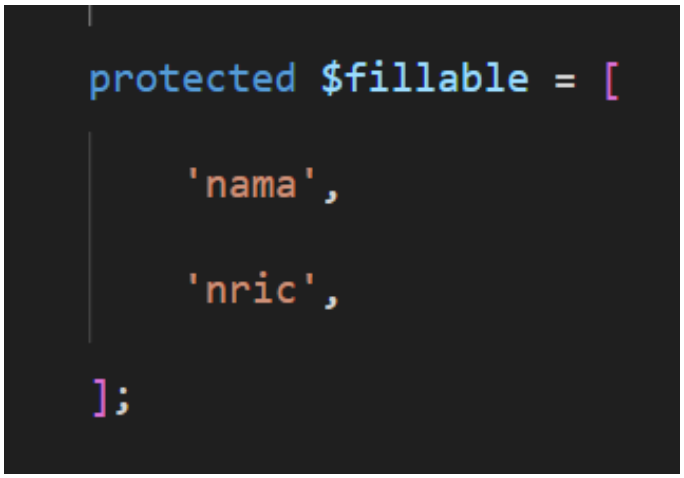

Figure 20: Add ‘nama’ and ‘nric’ Column

2.4.2.7 Add New Routes

Add the new routes for integration with MyDigital ID SSO in the **web.php** file to ensure proper routing for actions such as account login, callback and logout. Protected main route within the keycloak-web middleware to ensure that only authenticated users can access it.

```php
use Illuminate\Support\Facades\Route;

use Vizir\KeycloakWebGuard\Controllers\AuthController;


Route::get('/', function () {

    return view('welcome');

});


Route::group(['middleware' => 'keycloak-web'], function () {

    Route::get('/dashboard', function () {

        return view('dashboard');

    })->name('dashboard');

});


Route::get('/mydid-login', [AuthController::class, 'login'])->name('keycloak.login');

Route::get('/mydid-callback', [AuthController::class, 'callback'])->name('keycloak.callback');

Route::get('/mydid-logout', [AuthController::class, 'logout'])->name('keycloak.logout');
```

Figure 21: Add New Routes

2.4.2.8 Call Route Name

Call the route name that already defined in **web.php** to use in views.

```php
<button onclick="window.location.href='{{ route('keycloak.logout') }}'" class="button-1">Logout</button>

@else

<div class="card">

    <h2>No user information available.</h2>

    <button onclick="window.location.href='{{ route('keycloak.login') }}'" class="button-1">Login</button>
```

Figure 22: Call Route Name

2.4.3 PHP 8 with CodeIgniter Framework

This configuration guide is specifically applicable to **PHP v8.1.25** and **CodeIgniter framework v4.2.1**. It is important to note that this guide may not be universally compatible with all PHP and CodeIgniter versions. Developers should verify that their environment, including libraries, programming language, and framework versions, aligns with these requirements to ensure successful implementation.

2.4.3.1 Environment Configuration

Configure the keycloak configuration in the .**env** file:

```
KEYCLOAK_BASE_URL=MydigitalID_host

KEYCLOAK_REALM=mydid

KEYCLOAK_CLIENT_ID=Client ID

KEYCLOAK_CLIENT_SECRET=NUxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

KEYCLOAK_REDIRECT_URI=http://127.0.0.1:8000/callback
```

Figure 23: Environment Configuration in PHP Laravel

2.4.3.2 Library Installation

Install ***openid-connect-php*** library in CodeIgniter project folder.

```
composer require jumbojett/openid-connect-php
```

Figure 24: *openid-connect-php* Library Installation

2.4.3.3 Create *loginWithKeycloak* Function

Create a `loginWithKeycloak` function in the **Auth.php** file controller. Initialize the OpenID Connect client and set redirect URI. Authenticate the user and save user data in session to handle login logic.

```php
public function loginWithKeycloak()
{
    $oidc = new OpenIDConnectClient(
        getenv('KEYCLOAK_BASE_URL') . '/realms/' . getenv('KEYCLOAK_REALM'),
        getenv('KEYCLOAK_CLIENT_ID'),
        getenv('KEYCLOAK_CLIENT_SECRET')
    );
    $oidc->setRedirectURL(getenv('KEYCLOAK_REDIRECT_URI'));
    $oidc->authenticate();
    $userInfo = $oidc->requestUserInfo();
    $user = $this->auth_model->where('ic_number', $userInfo->nric)->first();
    if ($user) {
        session()->set('keycloak_user', [
            'nama' => $userInfo->nama, 'nric' => $userInfo->nric
        ]);
        foreach ($user as $k => $v) {
            session()->set('login_' . $k, $v);
        }
        return redirect()->to('/Main');
    } else {
        session()->set('keycloak_user', [
            'nama' => $userInfo->nama, 'nric' => $userInfo->nric
        ]);
        return view('auth/ic_number_not_found');
    }
```

Figure 25: *loginWithKeycloak* Function

2.4.3.4 Create *getAccessToken* Function

Create a *getAccessToken* function in the **Auth.php** file controller. This method is to exchange authorization code for access token.

```php
private function getAccessToken($code)
{
    $client = \Config\Services::curlrequest();
    $keycloakBaseUrl = getenv('KEYCLOAK_BASE_URL');
    $realm = getenv('KEYCLOAK_REALM');
    $response = $client->post("$keycloakBaseUrl/realms/$realm/protocol/openid-connect/token", [
        'form_params' => [
            'grant_type' => 'authorization_code',
            'client_id' => getenv('KEYCLOAK_CLIENT_ID'),
            'client_secret' => getenv('KEYCLOAK_CLIENT_SECRET'),
            'redirect_uri' => getenv('KEYCLOAK_REDIRECT_URI'),
            'code' => $code,
        ],
    ]);
        if ($response instanceof \CodeIgniter\HTTP\Response) {
        if ($response->getStatusCode() !== 200) {
            throw new \Exception("Failed to fetch access token from Keycloak: " . $response->getBody());
        }
        return json_decode($response->getBody(), true);
    }
    throw new \Exception("Invalid response from Keycloak");
}
```

Figure 26: getAccessToken Function

2.4.3.5 Create getUserInfo Function

Create a `getUserInfo` function in the Auth.php file controller. This method to retrieve user info using access token.

```php
private function getUserInfo($accessToken)
{
    $client = \Config\Services::curlrequest();
    $keycloakBaseUrl = getenv('KEYCLOAK_BASE_URL');
    $realm = getenv('KEYCLOAK_REALM');

    $response = $client->get("$keycloakBaseUrl/realms/$realm/protocol/openid-connect/userinfo", [
        'headers' => [
            'Authorization' => 'Bearer ' . $accessToken,
        ],
    ]);

    if ($response instanceof \CodeIgniter\HTTP\Response) {
        if ($response->getStatusCode() !== 200) {
            throw new \Exception("Failed to fetch user info from Keycloak: " . $response->getBody());
        }
        return json_decode($response->getBody(), true);
    }
    throw new \Exception("Invalid response from Keycloak");
}
```

Figure 27: *getUserInfo* Function

2.4.3.6 Create *keycloakCallback* Function

Create a `keycloakCallback` function in the **Auth.php** file controller. Fetch the authorization code and access token.

```php
public function keycloakCallback()
{
    try {
        $code = $this->request->getGet('code');
        if (!$code) {
            log_message('error', "Authorization code is missing.");
            throw new \Exception("Authorization code missing.");
        }
        log_message('debug', 'Authorization code received: ' . $code);

        $tokenData = $this->getAccessToken($code);
        log_message('debug', 'Access token retrieved: ' . json_encode($tokenData));

        $accessToken = $tokenData['access_token'];
```

Figure 28: keycloakCallback Function

Fetch user info and validate user information using the access token.

```php
$userInfo = $this->getUserInfo($accessToken);

log_message('debug', 'Keycloak user info retrieved: ' . json_encode($userInfo));


$nric = $userInfo['nric'] ?? null;

if (!$nric) {

    log_message('error', "NRIC missing in Keycloak user info.");

    throw new \Exception("NRIC is missing.");

}
```

Figure 29: Fetch userInfo

Store user data such as IC number and name in session.

```php
session()->set('keycloak_user', [

    'nama' => $userInfo['nama'] ?? 'Unknown',

    'nric' => $nric,

]);

log_message('debug', 'Keycloak User Data Stored in Session: ' . json_encode(session()->get('keycloak_user')));
```

Figure 30: Store User Data

Verify if the IC number matches in the database. If the IC number is found and matches, log the user in. Then, if the IC number is not found or does not match, handle linking or registration.

```
        $user = $this->auth_model->where('ic_number', $nric)->first();
        if ($user) {
            foreach ($user as $k => $v) {
                session()->set('login_' . $k, $v);
            }
            log_message('debug', 'Existing user logged in: ' . json_encode($user));
            return redirect()->to('/Main');
        } else {
            log_message('debug', 'User not found in DB. Redirecting to registration.');
            return view('auth/ic_number_not_found');
        }
    } catch (\Exception $e) {
        log_message('error', "Keycloak callback error: " . $e->getMessage());
        return redirect()->to('/login')->with('error', 'Authentication failed.');
    }
}
```

Figure 31: Verify IC Number

2.4.3.7 Add IC Number Column in Model

Add IC number column in the protected allowed field section in **Auth.php** file model.

```php
class Auth extends Model

{

    protected $DBGroup          = 'default';

    protected $table            = 'users';

    protected $primaryKey       = 'id';

    protected $useAutoIncrement = true;

    protected $insertID         = 0;

    protected $returnType       = 'array';

    protected $useSoftDeletes   = false;

    protected $protectFields    = true;

    protected $allowedFields    = ['name', 'password', 'ic_number'];
```

Figure 32: Add IC Number Column

2.4.3.8 Create *update_user* Function

Create an `update_user` function in the **Auth.php** file controller. Get IC number from session and find the user who has not linked their IC number yet. Update the user's IC number and log the user in and redirect to Main.

```php
public function update_user()
{
    $session = session();
    if ($this->request->getMethod() == 'post') {
        extract($this->request->getPost());
        $verify_password = password_verify($current_password, $session->login_password);
        if ($password !== $cpassword) {
            $session->setFlashdata('error', "Password does not match.");
        } elseif (!$verify_password) {
            $session->setFlashdata('error', "Current Password is Incorrect.");
        } else {
            $udata = [];
            $udata['name'] = $name;
            $udata['email'] = $email;
            if (!empty($password))
                $udata['password'] = password_hash($password, PASSWORD_DEFAULT);
            $update = $this->auth_model->where('id', $session->login_id)->set($udata)->update();
            if ($update) {
                $session->setFlashdata('success', "Your Account has been updated successfully.");
                $user = $this->auth_model->where("id ='{$session->login_id}'")->first();
                foreach ($user as $k => $v) {
                    $session->set('login_' . $k, $v);
                }
                return redirect()->to('/Main');
            } else {
                $session->setFlashdata('error', "Your Account has failed to update.");
            }
        }
    }
```

Figure 33: *update_user* Function

Retrieve the user information using the access token by querying the Keycloak User Info endpoint. Extract the IC number and name from the response to securely access and utilize these details within the application for user identification and processing.

2.4.3.9 Create *linkAccount* Function

Create a `linkAccount` function in the **Auth.php** file controller. Validate the email and password input. Attempt to authenticate the user using email and password. If the user is authenticated, update the IC number.

```php
public function linkAccount()
{
    $session = session();

    if ($this->request->getMethod() == 'post') {
        $email = $this->request->getPost('email');
        $password = $this->request->getPost('password');
        $user = $this->auth_model->where('email', $email)->first();

        if ($user && password_verify($password, $user['password'])) {
            $nric = session()->get('keycloak_user')['nric'];
            $this->auth_model->update($user['id'], ['ic_number' => $nric]);
            foreach ($user as $k => $v) {
                $session->set('login_' . $k, $v);
            }
            return redirect()->to('/Main');
        } else {
            $session->setFlashdata('error', 'Invalid email or password');
            return redirect()->back();
        }
    }
    return view('auth/ic_number_not_found');
}
```

Figure 34: linkAccount Function

2.4.3.10 Create *registerWithKeycloak* Function

Create a `registerWithKeycloak` function in the **Auth.php** file controller. Check if both IC number and name are available in the session. Create a new user if no existing user was found and redirect to the Main.

```php
public function registerWithKeycloak()
{
    You, 23 seconds ago • Uncommitted changes
    $keycloakUser = session()->get('keycloak_user');
    if (!$keycloakUser || empty($keycloakUser['nama']) || empty($keycloakUser['nric'])) {
        return redirect()->to('/login')->with('error', 'Session data is missing. Please log in again.');
    }
    $existingUser = $this->auth_model->where('ic_number', $keycloakUser['nric'])->first();
    if ($existingUser) {
        foreach ($existingUser as $k => $v) {
            session()->set('login_' . $k, $v);
        }
        return redirect()->to('/Main');
    }
    $newUserData = [
        'name' => $keycloakUser['nama'], 'password' => password_hash(bin2hex(random_bytes(8)), PASSWORD_DEFAULT), 'ic_number' => $keycloakUser['nric']
    ];
    if (!$this->auth_model->insert($newUserData)) {
        return redirect()->back()->with('error', 'Registration failed. Please try again.');
    }
    $user = $this->auth_model->where('ic_number', $keycloakUser['nric'])->first();
    foreach ($user as $k => $v) {
        session()->set('login_' . $k, $v);
    }
    return redirect()->to('/Main')->with('success', 'Registration successful.');
}
```

Figure 35: *registerWithKeycloak* Function

2.4.3.11 Create *logout* Function

Create a `logout` function in the **Auth.php** file controller. This logout process works by redirecting the user to the MyDigital ID logout endpoint along with the required OIDC logout request parameters such as id_token_hint, post_logout_redirect_uri, and client_id. These parameters ensure that the correct user session is terminated and that the user is redirected back to the application after logout.

```php
public function logout()
{
    $idToken = session()->get('keycloak_id_token');
    session()->destroy();
    if ($idToken) {
        $logoutUrl = getenv('KEYCLOAK_BASE_URL') . "/realms/" . getenv('KEYCLOAK_REALM') . "/protocol/openid-connect/logout?" .
        http_build_query([
            'id_token_hint' => $idToken,
            'post_logout_redirect_uri' => base_url('/'),
            'client_id' => getenv('KEYCLOAK_CLIENT_ID'),
        ]);
        return redirect()->to($logoutUrl);
    }
    return redirect()->to('/Auth/index');
}
```

Figure 36: *logout* Function

2.4.3.12 Add New Routes

Add new routes for all new functions related in **Routes.php** file.

```
$routes->get('/', 'Auth::index',['filter' => 'authenticated']);

$routes->get('/registerWithKeycloak', 'Auth::registerWithKeycloak',['filter' => 'authenticated']);

$routes->match(['post'], '/registerWithKeycloak', 'Auth::registerWithKeycloak',['filter' => 'authenticated']);

$routes->get('/Auth', 'Auth::index',['filter' => 'authenticated']);

$routes->get('/update_user', 'Auth::update_user',['filter' => 'authenticate']);

$routes->match(['post'], '/update_user', 'Auth::update_user',['filter' => 'authenticate']);

$routes->get('/Auth/(:segment)', 'Auth::$1',['filter' => 'authenticated']);

$routes->get('/login', 'Auth::index',['filter' => 'authenticated']);

$routes->match(['post'], '/login', 'Auth::index',['filter' => 'authenticated']);

$routes->get('/logout', 'Auth::logout');

$routes->match(['post'], '/auth/linkAccount', 'Auth::linkAccount');

$routes->get('/loginWithKeycloak', 'Auth::loginWithKeycloak');

$routes->get('/callback', 'Auth::keycloakCallback');
```

Figure 37: Add New Routes

2.4.4 PHP 7 with CodeIgniter Framework

This configuration guide is specifically applicable to **PHP v7.4.33** and **CodeIgniter framework v3.1.11**. It is important to note that this guide may not be universally compatible with all PHP and CodeIgniter versions. Developers should verify that their environment, including libraries, programming language, and framework versions, aligns with these requirements to ensure successful implementation.

2.4.4.1 Configuration Setup

Setup keycloak configuration in **config.php** file in Codeigniter 3 project folder.

```
$config['keycloak_base_url'] = 'MydigitalID_host';

$config['keycloak_realm'] = 'mydid';

$config['keycloak_client_id'] = 'Client ID';

$config['keycloak_client_secret'] = 'NUxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx';

$config['keycloak_redirect_uri'] = 'http://127.0.0.1:8000/callback';
```

Figure 38: Setup Configuration

2.4.4.2 Library Installation

Install ***oauth2-client*** library in CodeIgniter 3 project folder.

```
composer require league/oauth2-client
```

Figure 39: *oauth2-client* Library Installation

2.4.4.3 Create New Construct

Add a new construct in the **Oauth.php** file controller. Initialize OAuth 2.0 Client and set to return keycloak configuration.

```php
public function __construct() {
    parent::__construct();
    $this->load->model('admin_model','admin');
    $this->data['theme'] = 'admin';
    $this->data['model'] = 'login';
    $this->data['base_url'] = base_url();
    $this->load->helper('form');
    $this->data['csrf'] = array(
    'name' => $this->security->get_csrf_token_name(),
    'hash' => $this->security->get_csrf_hash()
    );

    $this->provider = new GenericProvider([
        'clientId'                => $this->config->item('keycloak_client_id'),
        'clientSecret'            => $this->config->item('keycloak_client_secret'),
        'redirectUri'             => $this->config->item('keycloak_redirect_uri'),
        'urlAuthorize'            => $this->config->item('keycloak_base_url') . '/realms/' . $this->config->item('keycloak_realm') . '/protocol/openid-connect/auth',
        'urlAccessToken'          => $this->config->item('keycloak_base_url') . '/realms/' . $this->config->item('keycloak_realm') . '/protocol/openid-connect/token',
        'urlResourceOwnerDetails' => $this->config->item('keycloak_base_url') . '/realms/' . $this->config->item('keycloak_realm') . '/protocol/openid-connect/userinfo',
    ]);
}
```

Figure 40: Add New Construct

2.4.4.4 Create *login* Function

Create a `login` function in the **Oauth.php** file controller. This method to get authorization URL by defining scope as openid and save the state for validation then redirect to Keycloak for login.

```php
public function login()
{
    $authorizationUrl = $this->provider->getAuthorizationUrl([
        'scope' => 'openid'
    ]);
    $this->session->set_userdata('oauth2state', $this->provider->getState());
    redirect($authorizationUrl);
}
```

Figure 41: *login* Function

2.4.4.5 Create *callback* Function

Create a `callback` function in **Oauth.php** file controller. Verify state parameters and get an access token using the authorization code grant.

```php
public function callback() {
  if (empty($this->input->get('state')) || ($this->input->get('state') !== $this->session->userdata('oauth2state'))) {
      $this->session->unset_userdata('oauth2state');
      show_error('Invalid state');
  }

  try {
      $accessToken = $this->provider->getAccessToken('authorization_code', [
          'code' => $this->input->get('code'),
      ]);
```

Figure 42: *callback* Function

Fetch user information and check if hashed IC number exists in the administrators table.

```php
$resourceOwner = $this->provider->getResourceOwner($accessToken);
$userData = $resourceOwner->toArray();
$nric = $userData['nric'];

$hashed_nric = hash('sha256', $nric);

$this->db->where('ic_number', $hashed_nric);
$user = $this->db->get('administrators')->row_array();
```

Figure 43: Fetch User Information

Verify if the hashed IC number matches in the database. If the IC number is found and matches, log the user in. Then, if the IC number is not found or does not match, handle linking or registration.

```
    if ($user) {
        $this->session->set_userdata('keycloak_user', $userData);
        $this->session->set_userdata('admin_id', $user['user_id']);
        $this->session->set_userdata('admin_profile_img', $user['profile_img']);
        $this->session->set_userdata('chat_token', $user['token']);
        $this->session->set_userdata('role', $user['role']);
        $access_result_data = $this->db->where('admin_id', $user['user_id'])->where('access', 1)->select('module_id')->get('admin_access')->result_array();
        $access_result_data_array = array_column($access_result_data, 'module_id');
        $this->session->set_userdata('access_module', $access_result_data_array);
        redirect('admin/dashboard');
    } else {
        $this->session->set_userdata('keycloak_temp_data', $userData);
        redirect('admin/oauth/show_modal_options');
    }
} catch (Exception $e) {
    show_error('Failed to authenticate with Keycloak: ' . $e->getMessage());
}
}
```

Figure 44: Verify IC Number

2.4.4.6 Create *show_modal_options* Function

Create `show_modal_options` function in **Oauth.php** file controller to load header, modal, and footer views.

```php
public function show_modal_options() {

    $this->data['theme'] = 'admin';

    $this->data['model'] = 'login';


    $this->load->view($this->data['theme'] . '/include/header', $this->data);

    $this->load->view('admin/login/modal_keycloak_options', $this->data);

    $this->load->view($this->data['theme'] . '/include/footer', $this->data);

}
```

Figure 45: show_modal_options Function

2.4.4.7 Create *link_existing_account* Function

Create `link_existing_account` function in **Oauth.php** file controller. Retrieve user info stored in session, validate the email and password input. Attempt to authenticate the user using email and password. If the user is authenticated, update the IC number.

```php
public function link_existing_account() {
    $this->data['theme'] = 'admin';
    $this->data['model'] = 'login';
    $keycloakData = $this->session->userdata('keycloak_temp_data');
    $nric = $keycloakData['nric'];
    if (!$keycloakData) {
        show_error('Session expired or Keycloak data not available.');
        return;
    }
    if ($this->input->post()) {
        $username = $this->input->post('username');
        $password = md5($this->input->post('password'));
        $this->db->where('username', $username);
        $this->db->where('password', $password);
        $user = $this->db->get('administrators')->row_array();
        if ($user) {
            $hashed_nric = hash('sha256', $nric);
            $this->db->where('user_id', $user['user_id']);
            $this->db->update('administrators', ['ic_number' => $hashed_nric]);
            $this->session->set_userdata('keycloak_user', $keycloakData);
            $this->session->set_userdata('admin_id', $user['user_id']);
            $this->session->set_userdata('admin_profile_img', $user['profile_img']);
            $this->session->set_userdata('chat_token', $user['token']);
            $this->session->set_userdata('role', $user['role']);
            $access_result_data = $this->db->where('admin_id', $user['user_id'])->where('access', 1)->select('module_id')->get('admin_access')->result_array();
            $access_result_data_array = array_column($access_result_data, 'module_id');
            $this->session->set_userdata('access_module', $access_result_data_array);
            redirect('admin/dashboard');
        } else {
            $this->session->set_flashdata('error', 'Invalid username or password');
            redirect('admin/oauth/link_existing_account');
```

Figure 46: *link_existing_account* Function

2.4.4.8 Create *register_new_account* Function

Create `register_new_account` function in **Oauth.php** file controller. Check if both IC number and name are available in the session. Create a new user if no existing user was found and redirected to the dashboard.

```php
public function register_new_account() {
    $keycloakData = $this->session->userdata('keycloak_temp_data');
    if (!$keycloakData) {
        show_error('Session expired or Keycloak data not available.');
        return;
    }
    $nric = $keycloakData['nric'];
    $email = isset($keycloakData['email']) ? $keycloakData['email'] : '';
    $username = isset($keycloakData['nama']) ? $keycloakData['nama'] : 'New User';
    $full_name = isset($keycloakData['nama']) ? $keycloakData['nama'] : 'New User';
    $defaultPassword = md5('defaultPassword123');
    $defaultRole = 1;
    $token = bin2hex(random_bytes(8));
    $hashed_nric = hash('sha256', $nric);
    $this->db->where('ic_number', $hashed_nric);
    $existingUser = $this->db->get('administrators')->row_array();
    if ($existingUser) {
        $this->session->set_userdata('keycloak_user', $keycloakData);
        $this->session->set_userdata('admin_id', $existingUser['user_id']);
        redirect('admin/dashboard');
```

Figure 47: *register_new_account* Function Part 1

```
    } else {
        $newUserData = [
            'username' => $username,
            'email' => $email,
            'ic_number' => $hashed_nric,
            'password' => $defaultPassword,
            'full_name' => $full_name,
            'role' => $defaultRole,
            'token' => $token
        ];
        if ($this->db->insert('administrators', $newUserData)) {
            $newUserId = $this->db->insert_id();
            $user = $this->db->where('user_id', $newUserId)->get('administrators')->row_array();
            $this->session->set_userdata('keycloak_user', $keycloakData);
            $this->session->set_userdata('admin_id', $user['user_id']);
            $this->session->set_userdata('admin_profile_img', $user['profile_img']);
            $this->session->set_userdata('chat_token', $user['token']);
            $this->session->set_userdata('role', $user['role']);
            $access_result_data = $this->db->where('admin_id', $user['user_id'])->where('access', 1)->select('module_id')->get('admin_access')->result_array();
            $access_result_data_array = array_column($access_result_data, 'module_id');
            $this->session->set_userdata('access_module', $access_result_data_array);
            redirect('admin/dashboard');
        } else {
            $error = $this->db->error();
            log_message('error', 'Database insert error: ' . print_r($error, true));
            show_error('Failed to insert new user data. Error: ' . $error['message']);
        }
    }
}
```

Figure 48: *register_new_account* Function Part 2

2.4.4.9 Create *logout* Function

Create a `logout` function in the **Oauth.php** file controller. This logout process works by redirecting the user to the MyDigital ID logout endpoint along with the required OIDC logout request parameters such as id_token_hint, post_logout_redirect_uri, and client_id. These parameters ensure that the correct user session is terminated and that the user is redirected back to the application after logout.

```php
    public function logout()
{
    $id_token = $this->session->userdata('id_token');
    $baseurl = $this->config->item('keycloak_base_url');
    $realm = $this->config->item('keycloak_realm');
    $client_id = $this->config->item('keycloak_client_id');
    $this->session->unset_userdata('admin_id');
    $this->session->unset_userdata('keycloak_user');
    $this->session->set_flashdata('success_message', 'Logged out successfully');
    if (!empty($id_token)) {
        $logoutUrl = $baseurl . '/realms/' . $realm . '/protocol/openid-connect/logout?' . http_build_query([
            'id_token_hint' => $id_token,
            'post_logout_redirect_uri' => base_url('admin'),
            'client_id' => $client_id,
        ]);
        redirect($logoutUrl);
    } else {
        redirect(base_url('admin'));
    }
}
```

Figure 49: *logout* Function

2.5 MyDigital ID Application Existence Check Module

The MyDigital ID SSO module cannot check the existence of the MyDigital ID application installed on the user's device because it is a WebView that has constraints to scan the mobile system environment. Therefore, we recommend that application developers that integrate with MyDigital ID SSO create a module to check the existence of the MyDigital ID application itself. We give two types of recommendations to application developers to provide awareness to application users that they must have a registered MyDigital ID application to smooth the user experience.

Developers may implement one of the following approaches, depending on platform and framework capabilities:

Option 1: App Presence Detection (Recommended Where Supported)

If the application framework allows detection of installed applications:

- The application **should detect** whether the MyDigital ID app is installed.
- If the app is not detected:
  i. Display a modal window, alert, or blocking notification.
  ii. Inform the user that a registered MyDigital ID application is required to proceed with login.
  iii. Prevent continuation of the MyDigital ID login process until resolved.

This approach provides a better user experience by preventing failed redirection attempts.

Option 2: Pre-Login Notice (For Frameworks with OS Restrictions)

If the application framework **does not support detection of installed apps**:

- The application should display a notice or alert before initiating MyDigital ID login.
- The notice must clearly inform users that:

i. A registered MyDigital ID application is required.

ii. The login process may fail if the app is not installed and registered.

This ensures user awareness even when technical detection is not possible.

All notices or alerts to include a direct link to download the MyDigital ID application:

Download URL:

https://www.digital-id.my/download

The link should:

- Open in the device's default browser.
- Be clearly labelled (e.g., "Download MyDigital ID App").
- Be easily accessible within the alert/modal.

2.5.1 Example Method To Detect MyDigital ID App In Mobile Application

iOS

To detect MyDigitalID app under IOS,

1. Add the following information in application's Info.plist

```
<key>LSApplicationQueriesSchemes</key>
<array>
<string>misignet</string>
</array>
```

2. Use canOpenURL function to with 'misignet://mydigitalid' as the url (parameter). Refer to https://developer.apple.com/documentation/uikit/uiapplication/canopenurl(_:) for more information on this function.

Android

1. Add the following section to the app's AndroidManifest.xml file

```
<queries>
    <package android:name="my.mimos.signetclient" />
</queries>
```

2. Call getInstalledApplications() or getInstalledPackages() and test for my.mimos.signetclient package name. Refer to https://developer.android.com/training/package-visibility for more information

React Native, Capacitor

For cross platform development, platform specific setting is required as mentioned in the corresponding platform section. Then, use `canOpenUrl` function for application detection. Refer to https://reactnative.dev/docs/linking#canopenurl and

https://capacitorjs.com/docs/apis/app-launcher#canopenurl for more information

Flutter

Create custom channel called `custom_launcher_channel` in both iOS and Android.

Add this in the *AppDelegate.Swift* for iOS.

```
import Flutter
import UIKit

@main
@objc class AppDelegate: FlutterAppDelegate {
 override func application(
  _ application: UIApplication,
  didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
 ) -> Bool {

   let controller : FlutterViewController = window?.rootViewController as! FlutterViewController
   let channel = FlutterMethodChannel(name: "custom_launcher_channel",
                                      binaryMessenger: controller.binaryMessenger)
```

```
  channel.setMethodCallHandler({
    (call: FlutterMethodCall, result: @escaping FlutterResult) -> Void in
    if call.method == "launchIntent" {
      if let args = call.arguments as? [String: Any],
        let urlStr = args["uri"] as? String,
        let url = URL(string: urlStr),
        UIApplication.shared.canOpenURL(url) {
        UIApplication.shared.open(url, options: [:], completionHandler: nil)
        result(true)
      } else {
        result(FlutterError(code: "INVALID_URL", message: "Cannot open URL", details: nil))
      }
    } else {
      result(FlutterMethodNotImplemented)
    }
  })

  GeneratedPluginRegistrant.register(with: self)
  return super.application(application, didFinishLaunchingWithOptions: launchOptions)
 }

 override func application(
  _ app: UIApplication,
  open url: URL,
  options: [UIApplication.OpenURLOptionsKey : Any] = [:]
 ) -> Bool {
  print("Received deep link: \(url.absoluteString)")
  return true
 }
```

Add this in the *MainActivity.kt* for Android.

```
package com.example.flutterexternalapp

import android.content.Intent
import android.net.Uri
import io.flutter.embedding.android.FlutterActivity
import io.flutter.embedding.engine.FlutterEngine
import io.flutter.plugin.common.MethodChannel
```

```
class MainActivity: FlutterActivity() {
 private val CHANNEL = "custom_launcher_channel"

 override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
  super.configureFlutterEngine(flutterEngine)
  MethodChannel(flutterEngine.dartExecutor.binaryMessenger, CHANNEL).setMethodCallHandler { call, result ->
   if (call.method == "launchIntent") {
    val uri = call.argument<String>("uri")
    try {
     val intent = Intent(Intent.ACTION_VIEW, Uri.parse(uri))
     intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
     startActivity(intent)
     result.success(null)
    } catch (e: Exception) {
     result.error("LAUNCH_ERROR", "Unable to launch intent", e.localizedMessage)
    }
   }
  }
 }
}
```

Call the method `custom_launcher_channel` to launch MyDigital ID app to launch app to existing app, and if no app was installed used the package `external_app_launcher` to open Google Play or Play Store. Refer to https://pub.dev/packages/external_app_launcher.

```
          if (uri != null) {
           final urlString = uri.toString();


           // Check for mydigital id app schemes to launch
           if (urlString.contains("misignet://") || urlString.contains("mydigitalid")) {
            if (await canLaunchUrl(uri)) {
              const platform = MethodChannel('custom_launcher_channel');
              await platform.invokeMethod('launchIntent', {'uri': urlString});
            } else {
              // Open App Store or Google Play
              await LaunchApp.openApp(
```

```
      androidPackageName: 'my.mimos.signetclient',
      iosUrlScheme: 'misignet://',
      appStoreLink: 'itms-apps://itunes.apple.com/us/app/mydigital-id/id1435289143',
    );
   }
   return NavigationActionPolicy.CANCEL;
  }
 }
```

Appendix

Appendix A: Application Survey Form

OP25073-MyDigitalID
_SSO_Application_Surv

Version History

Rev.	Revised Date	Change of History	Document Originator
0.1	22/11/2024	First draft.	My Digital ID Sdn. Bhd.
0.2	06/12/2024	Updated based on technical team review.	My Digital ID Sdn. Bhd.
0.3	16/01/2025	Third draft.	My Digital ID Sdn. Bhd.
0.4	24/1/2025	Fourth draft.	My Digital ID Sdn. Bhd.
0.5	28/1/2025	Fifth draft.	My Digital ID Sdn. Bhd.
0.6	13/2/2025	Sixth draft.	My Digital ID Sdn. Bhd.
0.7	4/3/2025	Seventh draft.	My Digital ID Sdn. Bhd.
0.8	25/3/2025	Eighth draft.	My Digital ID Sdn. Bhd.
1.0	27/3/2025	Baselined.	My Digital ID Sdn. Bhd.
1.1	14/7/2025	Update of Appendix A: Application Survey Form. Changes in Appendix A: Application Survey Form are as follows: i. Field “Description” ii. Field “Authentication Type”	My Digital ID Sdn. Bhd.
2.0	14/7/2025	2nd Baselined.	My Digital ID Sdn. Bhd.
2.1	14/7/2025	Update: i. Rebaselined into MyDigital ID SSO Integration Document. ii. No change in other content.	My Digital ID Sdn. Bhd.
3.0	14/7/2025	3rd Baselined.	My Digital ID Sdn. Bhd.
3.1	10/11/2025	Updates are as follow: i. Section 2.2 Library for Keycloak SSO Integration • Add and update list of libraries for Go, Dart and PHP ii. Section 2.3 – Keycloak Configuration • Update keycloak configuration example iii. Section 2.3.1 - Logout endpoint • Update Description and Example URL iv. Section 2.4 - Developer Guide for SSO Integration • Change PHP Code (update library, remove	My Digital ID Sdn. Bhd.

Rev.	Revised Date	Change of History	Document Originator
		prompt = login, add verify token)	
4.0	10/11/2025	4th Baselined.	My Digital ID Sdn. Bhd.
4.1	27/11/2025	Updates are as follows: i. Table 3: Keycloak Configuration Example ii. Table 4: Endpoint List	My Digital ID Sdn. Bhd.
5.0	27/11/2025	5th Baselined.	My Digital ID Sdn. Bhd.
5.1	9/2/2026	Newly added section 2.5 MyDigital ID Application Existence Check Module	My Digital ID Sdn. Bhd.
6.0	12/2/2026	6th Baselined.	My Digital ID Sdn. Bhd.

Review and Approval Record

i. My Digital ID Sdn. Bhd.

	Name	Title	Signatory	Date
Reviewed by (Technical):	Amru Yusrin Amruddin	Staff Engineer	AYA-Approve For Approval - SSO Integ	18/2/2026
Reviewed by (Technical):	Lee Chee Kiam	Principal Engineer	LCK-Approve For Approval - SSO Integ	12/2/2026
Reviewed by (QA):	Nurhumairak Radziah Abd Rahman	Senior Engineer	NRA-Approve For Approval - SSO Integ	12/2/2026
Approved by (QA Lead):	Rozaini Abdul Rahman	Principal Engineer	RAR-Approve For Approval - SSO Integ	16/2/2026
Approved by (Tech Lead):	Zahari Hadzir	Principal Engineer	ZH-Re For Approval - SSO Integration Guide	20/2/2026